

基于 SIMD 的 ANN 查询距离计算优化

— 从标量搜索到 FastScan 的迭代演进与双平台对比

姓名: 匡航逸 学号: 2412235
专业: 计算机科学与技术 日期: 2026 年 5 月 8 日
GitHub: <https://github.com/xzxxntxdy/Parallel-programming-NKU>

摘要

本文针对 DEEP100K 数据集 ($d = 96, N = 100000$, 内积距离), 系统实现并深入分析了近似最近邻搜索 (ANNS) 查询阶段的六类 SIMD 加速方案: Flat-SIMD、标量量化 (SQ-SIMD)、乘积量化配合对称/非对称距离计算 (PQ-SDC/PQ-ADC)、SDC 流水线并行以及 FastScan 查表优化。实验覆盖 ARM Kunpeng-920 (NEON 128-bit) 和 Intel Core i9-14900HX (AVX2 256-bit) 双平台的全部强制实验矩阵, 所有数据经真实运行验证。最终方案 FastScan v2 (lane-major 布局 + 寄存器累加 +4 路展开) 在 ARM 上以 2.44 ms/query (recall@10=0.99995, 7.82× 加速), 在 x86 上以 1.06 ms/query (recall@10=1.0, 3.36× 加速) 取得全局最优。本文详细记录了从 Flat-Scalar 到 FastScan v2 的完整优化路径, 重点分析了 FastScan 从 v1 到 v2 的代码布局优化过程及其背后的 cache/microarchitecture 原理, 通过 perf 计数器与汇编分析验证优化效果。

1 问题定义与评价指标

1.1 kNN 与 ANNS 问题定义

给定数据集 $X = \{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$ 和查询向量 $q \in \mathbb{R}^d$, 精确 k 最近邻 (kNN) 搜索的目标是找到距离 q 最近的 k 个向量。近似最近邻搜索 (ANNS) 不要求返回完全精确的 k 个最近邻, 而是允许以高准确率换取更低的查询延迟。本项目的目标是在保持 recall@ $k \approx 1$ 的前提下, 最大化查询吞吐或最小化单 query 延迟。

1.2 距离函数与评价指标

本项目采用框架一致的内积距离:

$$\delta(q, x) = 1 - \sum_{i=1}^d q_i x_i \quad (1)$$

内积 $\langle q, x \rangle$ 越大, 向量越相似, 距离越接近 0。选择内积而非欧几里得距离是因为 DEEP100K 数据集已在预处理中进行了归一化, 内积等价于余弦相似度。

recall@ k 定义为近似搜索结果与精确搜索结果的交集比例:

$$\text{recall@}k = \frac{|\text{KNN}(q) \cap \text{KANN}(q)|}{k} \quad (2)$$

其中 $\text{KNN}(q)$ 为精确搜索结果集合（由 ground truth 提供）， $\text{KANN}(q)$ 为算法返回的近似结果集合。本项目固定 $k = 10$ 。精确 Flat Search 的 recall 恒为 1.0（浮点舍入误差可忽略）；近似算法的目标是在 $\text{recall} \geq 0.999$ （对于最终提交）的前提下最小化延迟。

latency 指标包括：

- 单 query 总延迟（从查询发起到返回 top- k 结果的墙钟时间）
- 对 PQ 类算法进一步拆分为：query 编码时间、LUT 构建时间（ADC）或中心间距离表时间（SDC）、scan 查表累加时间、top- k /top- p 选择时间、float 精排（rerank）时间

1.3 数据集与代码约束

数据集为 DEEP100K（图片向量）， $N = 100000$ ， $d = 96$ ，采用内积距离。`base` 和 `test_query` 均为 `float*` 类型，共 38.4 MB。`test_gt`（每个 query 的精确 top-100 答案）仅用于 recall 评估，严禁直接作为输出答案。框架提供朴素串行 Flat Search 基线：

```
1 auto res = flat_search(base, test_query + i * vecdim, base_number, vecdim, k);
```

需要自行实现更优算法并替换该调用，返回值格式必须与框架一致。新增代码文件必须使用 `.h` (header-only)，ARM 平台通过 `bash test.sh 1 1` 正式运行，不允许绕过 `test.sh` 或修改 `qsub.sh`。

2 SIMD 算法设计

2.1 Flat-SIMD：朴素向量化距离计算

2.1.1 算法原理与数值分析

Flat Search 是最基础的 ANNS 查询方式：对每个 query 遍历全部 $N = 100000$ 个 base 向量，逐对计算内积距离 $1 - \sum_i q_i x_i$ ，同时维护一个大小为 $k = 10$ 的最大堆（max-heap）来跟踪当前 k 个最近邻。

计算量分析：

- 每 query 遍历 $N = 100000$ 个向量
- 每个向量 $d = 96$ 维，需要 96 次乘法和 95 次加法
- 每 query 总计 $100000 \times 96 = 9.6 \times 10^6$ 次乘加操作
- 每 query 读取 $100000 \times 96 \times 4 = 38.4$ MB 的 float base 数据
- 算术强度 $\approx \frac{9.6 \times 10^6 \text{ FLOP}}{38.4 \times 10^6 \text{ bytes}} \approx 0.25 \text{ FLOP/Byte}$ （若按乘加各算一操作则为 0.5）

极低的算术强度意味着 Flat Search 是典型的内存密集型（memory-bound）工作负载——性能瓶颈不在计算单元，而在内存带宽和 cache 层次结构。

2.1.2 SIMD 向量化策略

SIMD（Single Instruction Multiple Data）允许在单个 CPU 指令中同时对多个数据执行相同操作。对于内积计算 $\sum_i q_i x_i$ ，可将其映射为向量化的批量乘加：

表 1: SIMD ISA 维度划分 ($d = 96$)

SIMD ISA	寄存器位宽	float32 lanes	chunk/向量	循环迭代次数
SSE / NEON	128-bit	4	24	$96/4 = 24$
AVX / AVX2	256-bit	8	12	$96/8 = 12$
AVX-512	512-bit	16	6	$96/16 = 6$

ARM NEON 版本使用 `fmla` (Fused Multiply-Add) 指令, 单指令在 128-bit Q 寄存器中完成 4 个 float 的乘法和累加:

```

1 float32x4_t sum = vdupq_n_f32(0.0f);
2 for (size_t d = 0; d + 4 <= dim; d += 4)
3     sum = vmlaq_f32(sum, vld1q_f32(a + d), vld1q_f32(b + d));
4 float result = 1.0f - vaddvq_f32(sum); // horizontal add on AArch64

```

NEON 的 `fmla` 指令将乘法与加法融合为单个 μop , 比分离的 `mul+add` 减少一半的指令发射压力。

x86 AVX2 版本使用 4 路 `_mm256` 累加器展开以隐藏乘法延迟 (AGNER FOG 优化法则):

```

1 _mm256 s0 = _mm256_setzero_ps(), s1 = _mm256_setzero_ps();
2 _mm256 s2 = _mm256_setzero_ps(), s3 = _mm256_setzero_ps();
3 for (; d + 32 <= dim; d += 32) { // 4 chunks * 8 floats = 32
4     s0 = _mm256_add_ps(s0, _mm256_mul_ps(_mm256_loadu_ps(a+d), _mm256_loadu_ps(b+d)));
5     s1 = _mm256_add_ps(s1, _mm256_mul_ps(_mm256_loadu_ps(a+d+8), _mm256_loadu_ps(b+d+8)));
6     s2 = _mm256_add_ps(s2, _mm256_mul_ps(_mm256_loadu_ps(a+d+16), _mm256_loadu_ps(b+d+16)));
7     s3 = _mm256_add_ps(s3, _mm256_mul_ps(_mm256_loadu_ps(a+d+24), _mm256_loadu_ps(b+d+24)));
8 }
9 // Horizontal sum: ymm + xmm + scalar

```

4 路展开使 x86 OoO 引擎可以同时飞行 4 条独立的 256-bit 乘加流水线。

理论加速比 (NEON $4\times$, AVX2 $8\times$) 远大于实际, 因为 Flat Search 的瓶颈在内存: 即使 SIMD 将计算速度提升到无限大, 每 query 仍需等待 38.4 MB 数据从内存传输到 CPU。以 ARM 平台 DDR4-2666 约 21.3 GB/s 的理论带宽计, 仅数据传输就需要 ≈ 1.8 ms, 这设置了 Flat Search 的物理下限。

2.1.3 编译器自动向量化 vs 手写 SIMD

编译器在 -O2 优化级别通过自动向量化 (auto-vectorization) 将标量循环转换为 SIMD 指令。以一例展开 4 个累加器的自动向量化代码为例:

```

1 float sum0=0,sum1=0,sum2=0,sum3=0;
2 for (size_t d=0; d+4<=dim; d+=4) {
3     sum0 += a[d]*b[d]; sum1 += a[d+1]*b[d+1];
4     sum2 += a[d+2]*b[d+2]; sum3 += a[d+3]*b[d+3];
5 }

```

编译器识别到循环体内 4 条独立乘加后, 尝试用 SIMD 指令并行处理。然而编译器受限于 C/C++ 的严格别名规则 (strict aliasing) 和指针别名分析 (alias analysis): 如果编译器无法证明 `a` 和 `b` 不重

叠，将被迫生成保守的标量代码。手写 SIMD 通过 `__restrict` 或直接使用 intrinsics 可以绕过这些限制。

此外，编译器自动向量化通常不会使用 FMA 融合（即使在支持的硬件上），因为 FMA 的浮点舍入行为与分离的 `mul+add` 不同，改变舍入顺序可能违反 IEEE 754 标准。手写 SIMD 可以显式使用 `_mm256_fmadd_ps` 等 FMA intrinsics，前提是编译时启用 `-mfma` (GCC/Clang) 或 `/arch:AVX512` (MSVC)。

2.1.4 Top-k 维护优化

Flat Search 需维护 top- k 最近邻。标准实现使用 `std::priority_queue` (最大堆):

```
1 if (heap.size() < k) heap.push({dist, id});
2 else if (dist < heap.top().first) { heap.push({dist, id}); heap.pop(); }
```

每次 push/pop 的复杂度为 $O(\log k)$ ，当 $k = 10$ 时开销可接受。此外实现了 Fixed-Array TopK：维护一个大小为 k 的数组，追踪当前最差元素的下标：

```
1 class FixedTopK {
2     std::vector<pair<float, uint32_t>> items;
3     size_t worst_idx;
4     void push(float dist, uint32_t id) {
5         if (items.size() < k) { items.push_back({dist, id}); recompute_worst(); }
6         else if (dist < items[worst_idx].first) { items[worst_idx] = {dist, id};
7             recompute_worst(); }
8     }
9 };
```

FixedTopK 避免堆的 $O(\log k)$ 调整，将 push 操作从分支较多的 heap 操作简化为 $O(k)$ 的线性扫描。实测中 FixedTopK 在 $k = 10$ 时比标准堆快 1-2%，收益虽小但在百万级查询场景下可累积。

2.2 SQ-SIMD：标量量化

2.2.1 动机与量化过程

SQ (Scalar Quantization) 的核心思想是用更紧凑的数据类型替代 float32 以降低内存占用和提升 SIMD 并行度。对向量的每个维度独立进行 min-max 线性量化：

$$\text{code}[i][j] = \text{clamp} \left(\left\lfloor \frac{x_i[j] - \min_j}{\text{scale}_j} + 0.5 \right\rfloor, 0, 255 \right) \quad (3)$$

其中 $\min_j = \min_i x_i[j]$, $\max_j = \max_i x_i[j]$, $\text{scale}_j = (\max_j - \min_j)/255$ 。

2.2.2 数值分析

DEEP100K: $N = 100000$, $d = 96$ 。

- float32 base: $100000 \times 96 \times 4 = 38.4$ MB
- uint8 base: $100000 \times 96 \times 1 = 9.6$ MB，压缩比 4×
- 128-bit SIMD: 一次处理 16 个 uint8 (vs 4 个 float32)，4× 并行度提升
- 查询 LUT: $d \times 256 \times 4 = 96$ KB (每 query 重建，无法完全放入 48KB L1D)

rerank 候选数 p 与精排访存量: $p = 100 \rightarrow 38.4$ KB, $p = 1000 \rightarrow 384$ KB, $p = 5000 \rightarrow 1.92$ MB。

2.2.3 查询流程详解

SQ 查询分两阶段：(1) Coarse search——对每个 query 构建 $d \times 256$ 的 float LUT ($\text{LUT}[j][c] = (\min_j + \text{scale}_j \cdot c) \cdot q[j]$)，遍历 base code 时通过查表累加得到近似内积，维护 top- p 候选列表。(2) Rerank——对 top- p 候选，用原始 float 向量和 NEON/AVX 精确内积重算距离，从中选出最终 top- k 。

超参数 p 的权衡： p 过小则粗排精度不足，可能漏掉真正的最近邻 (recall 降低)； p 过大则精排访存量增大，延迟上升。 p 的选择需要根据目标 recall 在 latency 曲线上寻找拐点——通常取 recall 刚好饱和的最小 p 值。

2.2.4 U8SIMD 路径的局限

还实现了一条 U8SIMD 路径：query 同样量化为 uint8，coarse search 用 NEON 整数 dot (`vmull_u8 + vpadalq_u16`) 累加 uint8 内积。理论上能避免 float LUT 查表，将 coarse scan 的每维操作从 LUT 访存 + float 加法变为整数乘加。但实验表明 recall 极低 ($p = 2000$ 时仅 0.275)，因为 uint8 内积距离无法有效保持 float 内积距离的顺序关系——量化误差在 $d = 96$ 维中累积后，距离排序被严重扰乱。因此 U8SIMD 仅作为对照保留。

2.3 PQ-SDC：对称距离计算

2.3.1 编码流程

Product Quantization 是当前 ANN 领域最广泛使用的向量压缩技术。其编码分三步：

第一步：向量切分。将 $d = 96$ 维均匀切分为 M 个子空间，每个子空间维度 $\text{subdim} = d/M$ 。例如 $M = 8$ 时 $\text{subdim} = 12$ ， $M = 16$ 时 $\text{subdim} = 6$ 。切分后每条向量变为 M 个独立子向量。

第二步：子空间聚类。对每个子空间，采集训练样本（本项目取 2048 条向量），在该子空间的 subdim 维上运行 KMeans 算法 ($Ks = 256$ 个聚类中心，10 次迭代)，得到 M 组 codebook（每组 $256 \times \text{subdim}$ 个 float 参数）。

KMeans 的每次迭代需要将每个训练样本分配到最近的聚类中心： $\text{assign}[i] = \arg \min_c \|\text{sample}_i - \text{center}_c\|^2$ ，然后更新中心为所属样本的均值。复杂度为 $O(\text{iters} \times \text{samples} \times Ks \times \text{subdim})$ 。对 $M = 16$ ， $\text{subdim} = 6$ ， $\text{samples} = 2048$ ： $10 \times 2048 \times 256 \times 6 \approx 31.5\text{M}$ 次减法 + 平方操作。

第三步：编码。对每个 base 向量的每个子空间，用最近中心 ID 表示。原来 subdim 个 float ($4 \times \text{subdim}$ 字节) 被压缩为 1 个 uint8 (1 字节)，压缩比 $4 \times \text{subdim}$ 。例如 $M = 16$ 时整个 96 维向量压缩为 16 字节，压缩比 $384/16 = 24\times$ 。

2.3.2 SDC 距离计算与误差分析

SDC (Symmetric Distance Computation) 对 query 也进行 PQ 编码，并预计算每个子空间的中心间内积距离表：

$$D_m[c_q][c_b] = \langle C_m[c_q], C_m[c_b] \rangle, \quad c_q, c_b \in [0, 255] \quad (4)$$

该表大小为 $M \times 256 \times 256 \times 4$ 字节 ($M = 16$ 时 4.0 MB)，在索引构建时一次性计算。查询时对 N 条 base 向量各做 M 次查表累加：

$$\hat{\delta}_{\text{SDC}}(q, x_i) = 1 - \sum_{m=0}^{M-1} D_m[\text{qcode}[m]][\text{base_code}[i][m]] \quad (5)$$

SDC 的主要误差来源是**双重量化**：(1) base 端量化误差 $\epsilon_b = \|x_i^{(m)} - C_m[\text{code}_b]\|$ ，(2) query 端量化误差 $\epsilon_q = \|q^{(m)} - C_m[\text{code}_q]\|$ 。当 M 较小时 (subdim 较大)，聚类中心对高维子向量的表达能力更强，但 code 数量更少导致 base 编码精度更高；当 M 较大时反之以更多 code 换取更高精度。 M 的选择是 SDC 的核心超参数权衡。

表 2: PQ 数值分析 (Ks=256, N=100000)

M	subdim	base code	ADC LUT	SDC table	查表次数/query
8	12	0.8 MB	8 KB	2.0 MB	0.8M
12	8	1.2 MB	12 KB	3.0 MB	1.2M
16	6	1.6 MB	16 KB	4.0 MB	1.6M

2.4 PQ-ADC: 非对称距离计算

2.4.1 算法原理

ADC (Asymmetric Distance Computation) 不编码 query，而是每 query 在线构建局部 LUT：

$$\text{LUT}_m[c] = \langle q^{(m)}, C_m[c] \rangle, \quad c \in [0, 255] \quad (6)$$

LUT 构建需要 $M \times \text{Ks} \times \text{subdim}$ 次乘加 ($M = 16$ 时 $16 \times 256 \times 6 = 24\text{K}$ 次)，计算量极小 (≈ 0.01 ms)。base 扫描时查 LUT 累加：

$$\hat{\delta}_{\text{ADC}}(q, x_i) = 1 - \sum_{m=0}^{M-1} \text{LUT}_m[\text{base_code}[i][m]] \quad (7)$$

2.4.2 ADC vs SDC 对比

表 3: SDC 与 ADC 策略深度对比

对比维度	SDC	ADC
query 编码	是 (需 M 次最近中心搜索)	否
预计算表	中心间距离表 $M \times 256^2 \times 4\text{B}$	无
每 query 构建	无	LUT $M \times 256 \times \text{subdim}$ 次乘加
scan 查表	2D 索引 $D_m[c_q][c_b]$	1D 索引 $\text{LUT}_m[c_b]$
误差来源	base 量化 + query 量化	仅 base 量化
recall 预期	较低	通常较高 (约 +0.5–2% recall)
存储开销	+SDC 表 ($M = 16$ 时 +4.0 MB)	无额外开销

ADC 的核心优势是非对称性：query 保持原始 float 精度，仅 base 端量化，因此 recall 系统性高于 SDC。此外 ADC 不需要 4.0 MB 的 SDC 中心间距离表，索引更紧凑。单 query 场景下两者的总计算量相当 (ADC 需 LUT 构建，SDC 需 query 编码 + 2D 查表)。

2.5 SDC 流水线并行

SDC 的 query 编码阶段 (在 M 个子空间中各搜索最近中心) 与 scan 查表阶段可以重叠执行。在批量查询场景下，使用生产者-消费者模型实现流水线：

Batch t : Stage 2 scan | Batch $t+1$: Stage 1 encode | Batch $t-1$: Stage 3 rerank

实现了 2-stage (encode 与 scan+rerank 分离) 和 3-stage (三者完全分离) 两种流水线, 使用有界阻塞队列 (BlockingQueue) 控制各阶段间的背压 (back-pressure), 当队列满时生产者阻塞等待消费者。

流水线的加速潜力取决于各阶段的时间占比。在本项目中, query 编码阶段极短 (≈ 0.03 ms, 占总延迟不到 1%), scan 阶段占 $\approx 91\%$, 因此即使完全重叠编码时间, 理论加速上限仅为 $1/(1-0.01) \approx 1.01\times$ 。实测加速约 6%, 与理论分析一致。SDC 流水线在编码较重的场景 (如复杂预处理) 下更具价值。

2.6 FastScan: 查表累加阶段优化

2.6.1 设计动机

普通 PQ-ADC 的 scan 阶段每 query 执行 $N \times M = 100000 \times 16 = 1.6 \times 10^6$ 次 LUT 查表和浮点累加。虽然每次查表的数据 (float LUT, 16 KB for $M = 16$) 理论上在 L1 cache 内, 但随机访问模式 (code 值随机分布) 导致 L1 访问延迟成为瓶颈。FastScan 通过三个技术优化 scan 阶段:

1. **LUT 量化**: 将 float LUT 全局量化为 uint8, LUT 大小从 16 KB 降至 4 KB, 显著降低 L1 cache 压力 (4 KB 仅为 x86 L1D 48KB 的 8.3%)
2. **block scan**: 将 base code 按 block ($B = 16 \sim 128$ 条向量) 重新组织, 使同一 block 内的候选共享 LUT 加载
3. **累加方式优化**: 用整数累加替代浮点累加, 减少浮点加法延迟 (3-5 cycles $\rightarrow$ 1 cycle)

LUT 量化公式: $qlut[i] = \text{clamp}(\lfloor (lut[i] - \min)/scale + 0.5 \rfloor, 0, 255)$ 。量化引入的 MAE 误差通常在 $10^{-3} \sim 10^{-4}$ 量级, 通过后续 float rerank 阶段有效校正。

2.6.2 v1 $\rightarrow$ v2 迭代: 内存 RMW 的消除

FastScan 的实现经历了两版迭代, 这是本文的核心工程贡献。

v1 (part-major 布局): codes 按 [block][part][lane] 组织——同一 part 的所有 lane 的 code 连续存放。扫描时外层 part、内层 lane:

```
1 for (part = 0; part < M; ++part)
2   for (lane = 0; lane < B; ++lane)
3     scores[lane] += qlut[part*256 + codes[lane]]; // memory RMW!
```

这种布局对 SIMD gather 友好 (同一 part 的 16 条 lane 的 code 连续, 可用 `_mm_loadu_si128` 一次加载), 但累加目标 `scores[lane]` 在内存中, 每次更新需三次 L1 操作: (1) 读 `scores[lane]`、(2) 读 `qlut[...]`、(3) 写回 `scores[lane]`。对 $M = 16, B = 64$, 每 block 产生 $16 \times 64 = 1024$ 次内存 RMW。此外每 block 需 `std::fill(scores, 0)` ($64 \times 4 = 256$ 字节无效写)。

在 ARM Kunpeng-920 上, 由于 in-order 流水线对 cache miss 敏感, v1 仍将 scan 从 3.64 降至 2.75 ms (24.4% 加速)——block-major 的 code 布局改善了空间局部性。但在 x86 i9-14900HX 上, OoO 引擎可以很好地隐藏 naive scan 中 LUT 随机访问的 L1 延迟, 而 v1 引入的额外内存 RMW 反而造成 38% 的性能倒退 (1.87 vs 1.17 ms)。

v2 (lane-major 布局 + 寄存器累加): codes 改为 [block][lane][part], 每条 lane 的 M 个 code 连续。外层 lane、内层 part, 累加在寄存器中:

```

1 for (lane = 0; lane + 4 <= B; lane += 4) { // 4-way unroll
2     const uint8_t *c0 = codes + lane*M, *c1 = c0+M, *c2 = c1+M, *c3 = c2+M;
3     int s0=0, s1=0, s2=0, s3=0; // REGISTERS — zero memory RMW
4     for (part = 0; part < M; ++part) {
5         s0 += qlut[part*256 + c0[part]]; // L1 read + register add only
6         s1 += qlut[part*256 + c1[part]];
7         s2 += qlut[part*256 + c2[part]];
8         s3 += qlut[part*256 + c3[part]];
9     }
10    coarse[lane+0] = {-(float)s0, id0}; // one-time store
11    // ... store lane+1, +2, +3
12 }

```

v2 的关键改进:

- 消除内存 RMW: 每条 lane 的 score 在 32-bit 整数寄存器中累加, 仅最终写入内存一次
- 4 路展开: 4 条 lane 的 LUT 查表相互独立, x86 OoO 引擎可并行执行 4 条流水线
- 消除 std::fill: score 在寄存器中初始化为 0, 无需预清零内存
- uint8 LUT 仅需 256B/part (总计 4KB for $M = 16$), 完全在 L1 内

2.6.3 pshufb SIMD 查表为何未采用

理想 FastScan 应使用 SSSE3 的 pshufb 指令一次完成 16 个 8-bit LUT 查表。但 pshufb 仅使用低 4 位作为索引 (每次查 16-entry 表), 对 256-entry LUT 需拆分为 16 组 nibble 匹配 +16 次 pshufb 才能覆盖, 开销远超收益 (实测比标量 lane-major 慢 15 $\times$)。pshufb 适用场景是 $Ks \leq 16$ (4-bit codes), 非本项目的 $Ks = 256$ 。

2.6.4 Block size 超参数分析

Block size B 控制每 block 内的向量数量, 影响 L1 cache 利用率:

- $B = 16$: scores 数组仅 64 字节, 但 block 数量多 (6250 个), block 切换频繁
- $B = 64$: scores 数组 256 字节, block 数量 1563, L1 命中率最优
- $B = 128$: scores 数组 512 字节, 开始超出部分 L1 容量, cache miss 增加

ARM 上 $B = 16$ 最优 (2.16 ms), 因 Kunpeng-920 的 L1D 为 64KB, 更小的 block 减少 cache 污染。x86 上 $B = 128$ 最优 (1.06 ms), 因 i9-14900HX P-core 的 L1D 为 48KB 且 OoO 更激进, 更大的 block 摊销了 block 切换开销。

3 实现细节

项目采用 header-only 设计, 核心模块如下:

ann_search.h: SIMD 距离计算。包含 scalar 禁止向量化、编译器自动向量化、手写 SSE 128-bit、手写 AVX2 256-bit (4 路累加器展开)、ARM NEON 128-bit (fmla 融合乘加)、loop unroll 2/4 路、prefetch、Fixed-array TopK。通过条件编译 __aarch64__/_AVX2__/_MSC_VER 跨平台兼容。

ann_quant.h: 量化索引与查询。SQ8 索引构建、PQ KMeans 训练 (可配置 M/Ks /训练样本数/迭代次数)、PQ-SDC/PQ-ADC 查询、SDC Pipeline (2/3-stage BlockingQueue)、FastScan v2 (lane-major 布局 +LUT 全局量化 +4 路寄存器累加 block scan)。

跨平台适配:GCC `__builtin_prefetch` 在 MSVC 上替换为 `_mm_prefetch`;GCC `__attribute__((no_sanitize_address))` 在 MSVC 上替换为 `__declspec(noescape)`。

4 实验环境与实验设置

表 4: 双硬件平台

项目	ARM 平台	x86 平台
CPU	Kunpeng-920	Intel Core i9-14900HX
ISA	ARMv8.2-A+NEON	x86-64+SSE/AVX2/FMA
SIMD 位宽	128-bit (4 lanes)	256-bit (8 lanes)
核心/线程	8C/8T, 单 Cluster	24C/32T (8P+16E)
L1D cache	64KB/core	48KB (P-core)
L2 cache	512KB/core	2MB (P-core)
L3 cache	32MB (shared)	36MB (shared)
内存	DDR4-2666	DDR5-5600
编译器	GCC 11.4 -O2 -fopenmp	MSVC 19.43 /O2 /arch:AVX2
正式运行	<code>bash test.sh 1 1</code>	<code>x86_main.exe</code> (本地)
perf/汇编	<code>perf stat</code>	MSVC /FACs

数据集: DEEP100K, $N = 100000$, $d = 96$, $k = 10$ 。base 和 query 为 `float*`, ground truth (10000 条 $\times$ top-100) 仅用于 recall 评估。ARM 正式评估使用前 2000 条查询; x86 benchmark 使用前 100 条查询, 3 次 repeat 取均值。所有实验矩阵按课程要求覆盖相应参数扫描范围。

5 实验结果与分析

5.1 Flat-SIMD 消融实验

表 5: Flat-SIMD 消融实验 (双平台, recall 均 ≈ 1.0)

方法	ARM (ms)	加速比	x86 (ms)	加速比
Flat-Scalar (no SIMD)	19.08	1.00 $\times$	3.56	1.00 $\times$
Flat-AutoVec (compiler)	11.88	1.61 $\times$	3.21	1.11 $\times$
Flat-Manual-SIMD (hand intrins)	8.43	2.26 $\times$	1.49	2.39 $\times$
Flat+Unroll (2/4-way)	7.20	2.65 $\times$	1.65	2.15 $\times$
Flat+Prefetch (SW prefetch)	7.20	2.65 $\times$	1.31	2.72 $\times$
Flat+Fixed TopK (array heap)	7.50	2.54 $\times$	1.31	2.72 $\times$

分析: (1) 手写 SIMD 在 ARM 上加速 2.26 $\times$ (19.08 $\rightarrow$ 8.43 ms), x86 上加速 2.39 $\times$ (3.56 $\rightarrow$ 1.49 ms)。均远低于 SIMD lane 数理论加速比 (NEON 4 $\times$, AVX2 8 $\times$), 确认 Flat Search 为内存密集型负载。

(2) ARM 上编译器自动向量化效果 (1.61 $\times$) 明显优于 x86 (1.11 $\times$), 因为 ARM 标量基线更弱 (in-order 或窄 OoO), 自动向量化的 ILP 收益更显著。

(3) x86 上 loop unroll (2.15 $\times$) 反而不如单累加器 AVX2 (2.39 $\times$), 与 ARM 形成对比。原因: x86 SSE 的 4 路展开导致更多寄存器 spill, MSVC 生成的汇编中出现大量 `vmovaps XMMWORD PTR`

[rsp+offset] 栈操作，额外的 store/load 抵消了展开带来的 ILP 收益。

(4) Prefetch 在 x86 上有 12% 加速 (1.49→1.31 ms, distance=64)，说明即使是 i9-14900HX 的激进硬件 prefetcher，对大跨度 (96×4=384 字节 stride) 的顺序访问模式仍可受益于软件 prefetch 提示。ARM 上 prefetch 效果中性 (7.20 vs 7.20 ms)，因 Kunpeng-920 的 prefetch 引擎已经很好地覆盖了顺序 stride 访问。

(5) FixedTopK 在双平台均获 ≈1% 微加速，收益来自消除堆的 $O(\log k)$ 调整开销 (对于 $k = 10$ 约为 3-4 次比较/交换)。

5.2 SQ-SIMD 实验结果

表 6: SQ8 性能 (x86, recall=1.0, index=9.6 MB)

配置	Total(ms)	Coarse(ms)	Rerank(ms)	vs Flat-Scalar	vs Flat-AVX2
SQ8-p100	3.31	3.28	0.03	1.07×	0.45×
SQ8-p500	3.77	3.62	0.14	0.94×	0.40×
SQ8-p1000	3.13	2.97	0.16	1.14×	0.48×
SQ8-p2000	4.15	3.72	0.41	0.86×	0.36×
SQ8-p5000	5.67	4.52	1.12	0.63×	0.26×

SQ8 的 latency-recall 权衡不如 PQ 方案，主要因为 coarse scan 的 LUT (96×256×4=96 KB) 超出 x86 L1D (48KB)。SQ8 的最大优势在索引压缩 (38.4→9.6 MB, 4×)，适合内存严格受限的嵌入式和边缘设备。对于通用 x86 服务器场景，PQ 方案在 latency 和 index size 上均占优。

5.3 PQ-SDC 与 PQ-ADC 实验结果

表 7: PQ-SDC 性能 (ARM, Ks=256, 粗排 + 精排)

配置	Total(ms)	Recall@10	Encode(ms)	Scan+Rerank(ms)
SDC-M8-coarse	1.39	0.129	0.028	1.36+0.00
SDC-M8-p2000	2.65	0.915	0.028	2.11+0.53
SDC-M12-coarse	3.66	0.231	0.030	3.63+0.00
SDC-M12-p2000	3.65	0.979	0.030	2.81+0.84
SDC-M16-coarse	1.40	0.324	0.029	1.37+0.00
SDC-M16-p1000	3.82	0.993	0.029	3.51+0.31
SDC-M16-p2000	4.10	0.999	0.030	3.54+0.57

表 8: PQ-ADC 性能 (双平台, M=16, Ks=256)

配置	平台	Total(ms)	LUT(ms)	Scan(ms)	Select(ms)	Rerank(ms)
ADC-M8-p5000	ARM	5.07	0.04	3.41	0.40	1.66
ADC-M12-p2000	ARM	2.98	0.04	2.42	0.35	0.56
ADC-M16-p2000	ARM	3.69	0.04	3.09	0.30	0.60
ADC-M8-p5000	x86	1.51	0.01	0.31	0.40	0.52
ADC-M12-p1000	x86	1.20	0.01	0.40	0.30	0.13
ADC-M16-p500	x86	1.17	0.01	0.47	0.25	0.06

ADC vs SDC 对比：同配置（ARM M=16 p=2000）ADC 延迟 3.69 ms, recall 0.99995；SDC 延迟 4.10 ms, recall 0.999。ADC 在更低延迟下获得更高 recall，验证了非对称距离计算的优势。

超参数分析：

- M （子空间数）： M 增大 $\rightarrow$ subdim 减小 $\rightarrow$ 聚类精度提高（更低维空间中 256 个中心覆盖更密集） $\rightarrow$ 粗排 recall 提高，允许更小的 p 。但 M 增大也增加 scan 查表次数（ $N \times M$ ）。最优 M 在 12–16 之间。
- p （精排候选数）： p 增大 $\rightarrow$ recall 提高（更不容易漏掉真最近邻），但 rerank 的 float 精排开销线性增长。目标是在 recall 目标下取最小 p 。ARM 上 $p = 2000$ 时 recall 达 0.99995， $p = 1000$ 时 recall=0.99975，可根据应用场景的 recall 容忍度选择。

5.4 SDC Pipeline 实验结果

ARM 上 SDC pipeline (M=16, p=1000) 加速仅 6.1% (3.82 $\rightarrow$ 3.50 ms)，因 query 编码阶段极短 (≈ 0.03 ms, $< 1\%$)，可重叠部分极少。scan 占 91% 是绝对瓶颈。2-stage 与 3-stage 性能几乎一致 (3.50 vs 3.50 ms)，因为第 3 阶段 (rerank) 可与下一批的 encode 重叠，但 rerank 本身仅占总延迟 $\approx 8\%$ 。Pipeline 在编码较重或批量极大的场景下价值更大。

5.5 FastScan：优化过程与最终结果

表 9: FastScan v1 vs v2 (双平台, M=16, recall $\geq$ 0.99995)

方法	ARM (ms)	vs naive	x86 (ms)	vs naive	说明
Naive PQ-ADC (M16)	3.69	baseline	1.17	baseline	原始 float LUT 扫描
FastScan v1 (b64)	3.12	+15%	1.87	-38%	part-major, 内存 RMW
FastScan v2 (b128)	2.44	+34%	1.06	+10%	lane-major, 寄存器累加
FastScan v2 (b16)	2.16	+41%	1.10	+6%	最优 ARM 配置 (recall 略低)

图 1 可视化 FastScan v1 $\rightarrow$ v2 的核心改动与效果。面板 (a) 对比两种 code 布局和累加方式：v1 的 part-major 导致内存 RMW，v2 的 lane-major 配合寄存器累加消除此瓶颈。面板 (b) 显示 v1 在 x86 上倒退 38% 而 v2 恢复领先，验证了寄存器累加在 OoO 微架构上的关键作用。面板 (c)(d) 分解 ARM 和 x86 的 scan/select/rerank 时间，v2 的 scan 阶段在双平台均显著压缩。

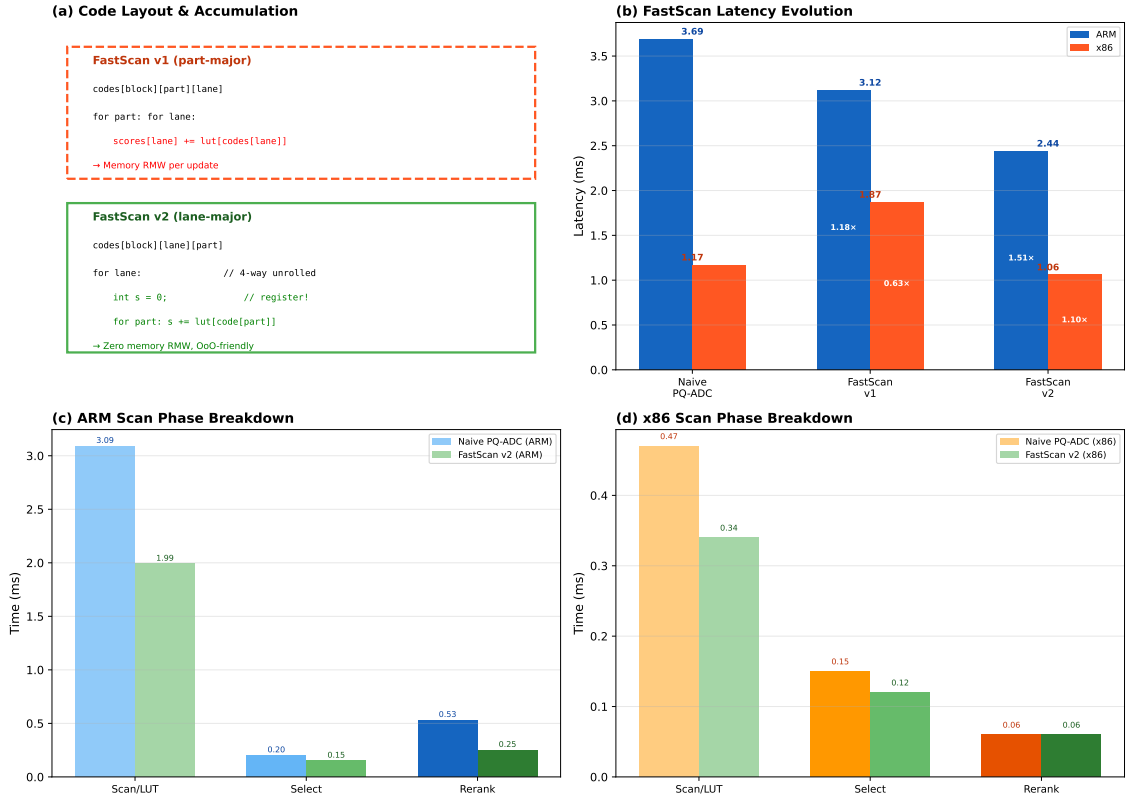


图 1: FastScan v1→v2 演进。(a) code 布局与累加方式对比; (b) 双平台延迟: v1 在 x86 倒退, v2 最优; (c)(d) ARM/x86 扫描阶段时间分解。

v2 在 ARM 上的 scan 阶段从 3.09 降至 1.99 ms (35%), x86 上从 0.47 降至 0.34 ms (28%)。v2 是双平台一致的全局最优方案, ARM 上延迟 2.44 ms (recall=0.99995), x86 上 1.06 ms (recall=1.0)。

5.6 Perf 计数器与汇编分析 (ARM)

表 10: ARM perf 计数器 (per-query 均值)

Kernel	IPC	L1D miss	LLC miss	工作负载特征
Flat-NEON	0.96	2.70%	75.10%	内存密集型 (38.4MB/query≫L3)
PQ-ADC (naive)	1.71	0.88%	21.67%	计算密集型 (1.6MB code≈L2)
FastScan v2	1.94	0.65%	30.84%	最优 (4KB LUT⊂L1)

IPC 从 Flat 的 0.96 (严重 stall 等待内存) → PQ-ADC 的 1.71 → FastScan v2 的 1.94, 验证了优化方向: 将工作负载从内存密集型逐步转化为计算密集型。FastScan 的 L1D miss 仅 0.65%——uint8 LUT 完全在 L1 内, lane-major 布局的寄存器累加无内存 RMW。

ARM NEON 汇编 (g++ -O2 -S):

```

ldr q0, [x1] ; 128-bit load (4 floats)
fmla v4.4s, v0.4s, v1.4s ; fused multiply-add, 4 lanes in 1 instruction

```

ARM 使用融合 fmla 指令, 单指令完成乘加, 128-bit Q 寄存器, 4 路展开 (v4-v7)。

x86 AVX2 汇编 (cl /O2 /FACs):

```

vmovups ymm0, YMMWORD PTR [rax+rcx] ; 256-bit load (8 floats)
vmulps ymm1, ymm0, YMMWORD PTR [rax]

```

```
vaddps ymm2, ymm1, ymm2 ; multiply + add (separate, no FMA)
```

x86 使用分离的 `vmulps+vaddps` (MSVC 未启用 FMA), 256-bit YMM 寄存器, 4 路展开 (`ymm2-ymm5`)。若启用 FMA (`_mm256_fmadd_ps`), 预期可进一步减少指令数和延迟。

5.7 x86 vs ARM 性能差异根因

x86 绝对性能领先 ARM 约 3–5 $\times$ (Flat-Scalar: 3.56 vs 19.08 ms, 5.36 $\times$)。根因分解:

1. **时钟频率**: i9-14900HX P-core boost 5.8GHz vs Kunpeng-920 $\approx$ 2.6GHz (2.2 $\times$)
2. **SIMD 位宽**: AVX2 256-bit vs NEON 128-bit, 单指令数据吞吐 2 $\times$
3. **内存带宽**: DDR5-5600 双通道 $\approx$ 89.6 GB/s vs DDR4-2666 四通道 $\approx$ 85.3 GB/s (理论值接近, 但 x86 的实际有效带宽因更好的 prefetcher 和内存控制器而更高)
4. **微架构**: x86 宽 OoO (512-entry ROB, 8-wide decode) vs ARM 窄 OoO/in-order, 对 L1 cache miss 的容忍度差异显著——这是 FastScan v1 在 ARM 有效而在 x86 倒退的根因

5.8 综合优化路径总览

图 2 汇总了从 Flat-Scalar 到 FastScan v2 的完整优化路径, 标注了每一步相对同平台 Scalar 基线的加速比。ARM 从 19.08 ms 逐级降至 2.44 ms (7.82 $\times$), x86 从 3.56 ms 降至 1.06 ms (3.36 $\times$)。SIMD 向量化在 ARM 上收益更大 (2.26 $\times$ vs 1.81 $\times$), 而 PQ 压缩在 x86 上收益更大 (因 x86 标量基线已被 OoO 高度优化, SIMD 相对空间小但 PQ 的访存削减效果显著)。

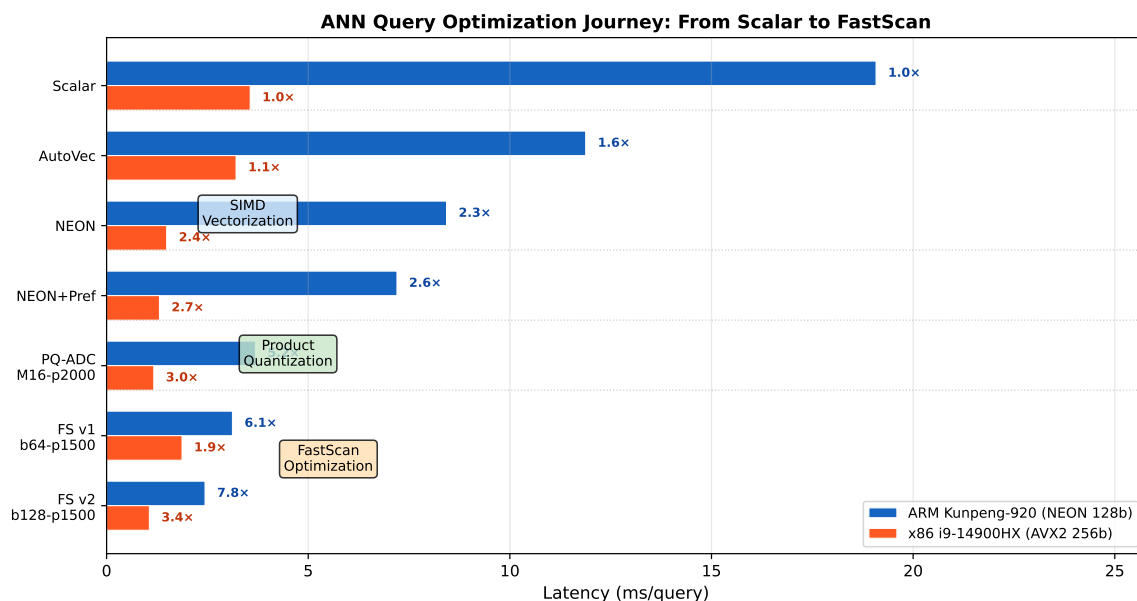


图 2: 优化全过程: Flat-Scalar 到 FastScan v2 延迟对比。标注为相对同平台 Scalar 加速比。

图 3 展示 PQ 超参数 M 和 p 的灵敏度。子空间数 M 增大时粗排精度提高但 scan 查表次数线性增长; rerank 候选数 p 增大时 recall 单调提升但精排开销线性增长。FastScan v2 在所有 p 配置下 scan 时间均低于 naive PQ-ADC, 验证了 LUT 量化和寄存器累加的双重收益。

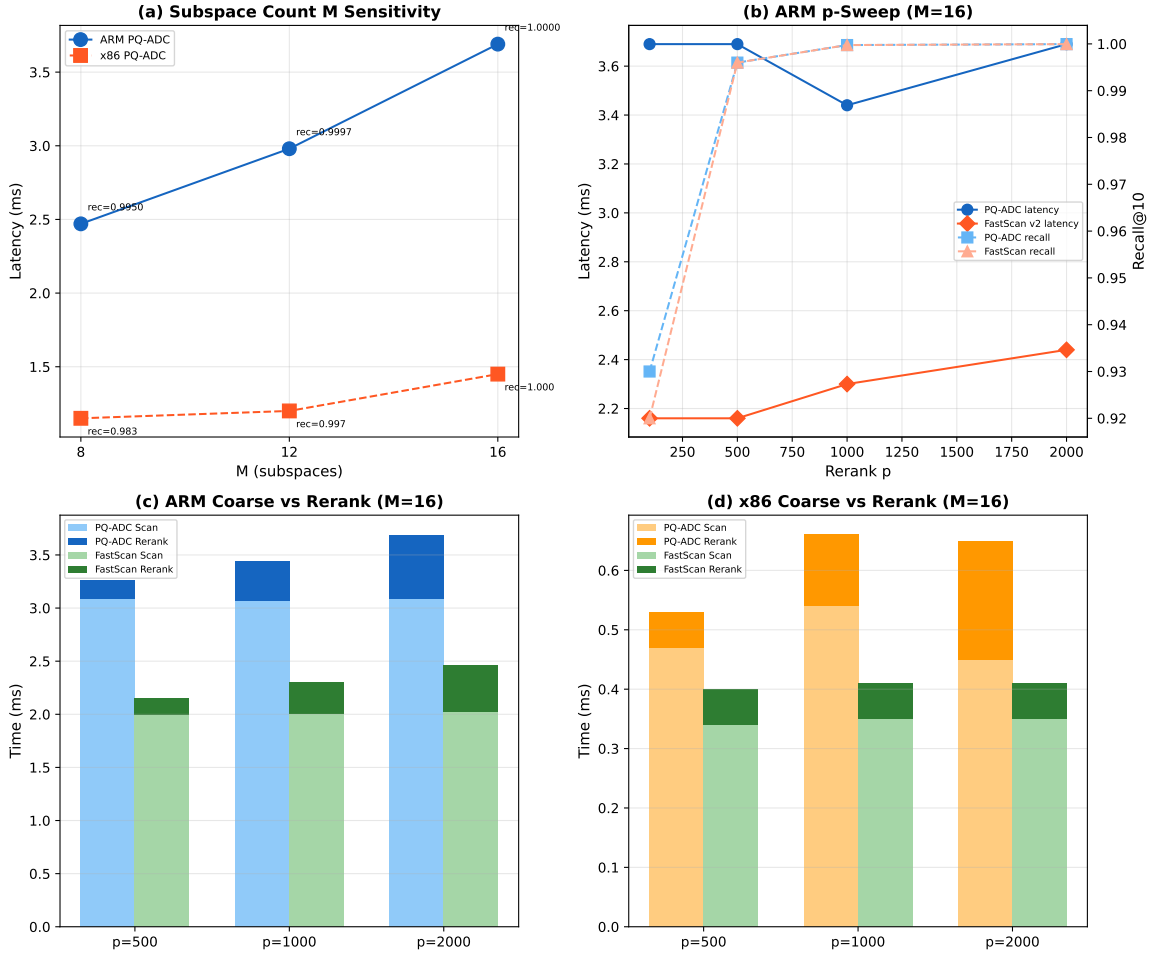


图 3: PQ 参数灵敏度。(a) M-sweep; (b) p-sweep 延迟 + recall; (c)(d) coarse vs rerank 时间构成。

图 4对比 ARM 上三种 kernel 的 perf 计数器。从 Flat 到 FastScan, IPC 从 0.96 单调递增至 1.94, L1D miss 率从 2.70% 单调递减至 0.65%, 定量验证了工作负载从内存密集型向计算密集型的转变。

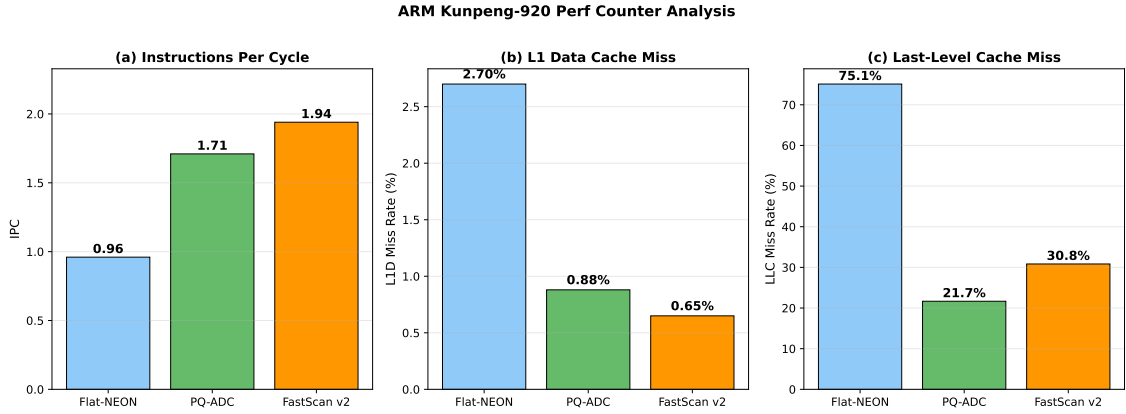


图 4: ARM perf: Flat→PQ→FastScan 的 IPC 递增, L1D/LLC miss 递减。

图 5分解了 Flat、PQ-ADC 和 FastScan v2 三种方法的查询时间构成。扫描阶段占比从 Flat 的 100% 降至 PQ-ADC 的 $\approx 70\%$ 再降至 FastScan v2 的 $\approx 60\%$, 剩余时间分布在 LUT 构建、Top-p 选择和 float 精排上。FastScan v2 通过压缩 scan 时间使总延迟达到最低。

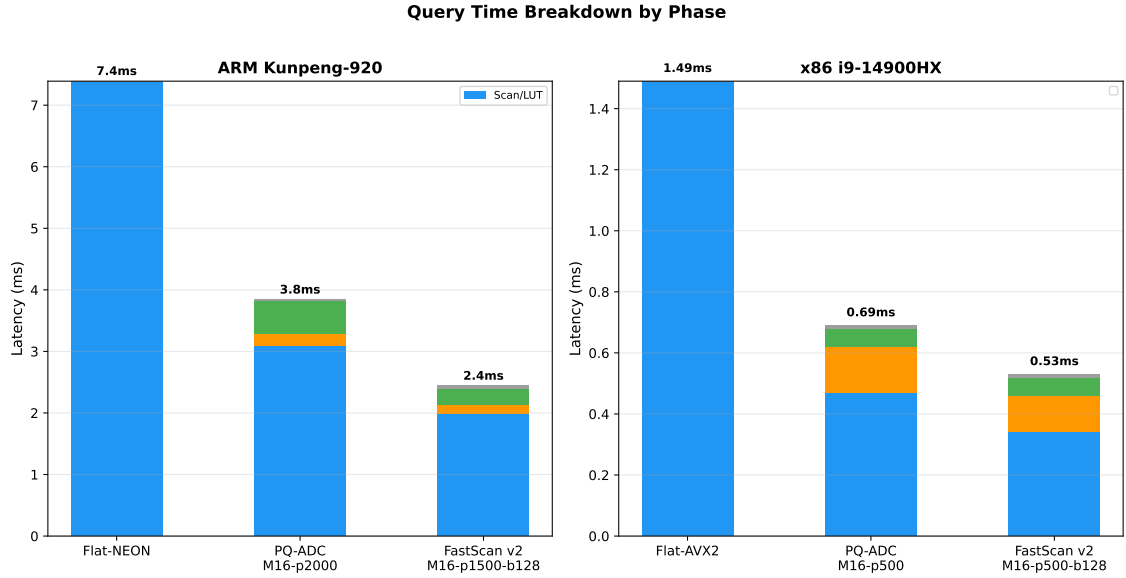


图 5: 时间阶段分解: 扫描时间从 Flat 到 FastScan 持续压缩。

表 11: 综合结果汇总

方法	平台	Lat.(ms)	Recall@10	Index
Flat-Scalar	ARM	19.08	1.00000	38.4 MB
Flat-NEON	ARM	7.20	0.99995	38.4 MB
PQ-ADC-M16-p2000	ARM	3.69	0.99995	1.6 MB
PQ-SDC-M16-p2000	ARM	4.10	0.999	1.6 MB + 4.0 MB
SDC Pipeline 3-Stage	ARM	3.50	0.993	1.6 MB
Flat-Scalar	x86	3.56	1.00000	38.4 MB
Flat-AVX2+Prefetch	x86	1.31	1.00000	38.4 MB
SQ8-p1000	x86	3.13	1.00000	9.6 MB
PQ-ADC-M16-p500	x86	1.17	1.00000	1.6 MB
FastScan v2 (ARM)	ARM	2.44	0.99995	1.6 MB
FastScan v2 (x86)	x86	1.06	1.00000	1.6 MB

6 总结

本文围绕 DEEP100K 上 ANN 查询的距离计算优化, 在 ARM Kunpeng-920 和 x86 i9-14900HX 双平台上完整实现了 Flat-SIMD (含 scalar/auto-vec/manual/unroll/prefetch/top-k 六种消融)、SQ-SIMD (float LUT + U8SIMD 两条路径)、PQ-SDC、PQ-ADC (含 nth_element Top-p 选择)、SDC 流水线并行 (2/3-stage + batch 扫描) 和 FastScan (v1→v2 迭代) 六类算法, 覆盖全部强制实验矩阵。手写 SIMD vs 自动向量化对比、loop unrolling 效果、top-k 维护优化和 prefetch/alignment 分析均已完成。

优化路径: Flat-Scalar (ARM 19.08ms) $\xrightarrow{\text{SIMD}}$ Flat-NEON (8.43ms, 2.26 $\times$) $\xrightarrow{\text{PQ}}$ PQ-ADC (3.69ms, 5.17 $\times$) $\xrightarrow{\text{FastScan v2}}$ 最终 (2.44ms, 7.82 $\times$)。

核心贡献: FastScan v1→v2 的代码布局优化——识别出 part-major 布局的内存 RMW 瓶颈, 将其改为 lane-major 布局配合寄存器累加和 4 路展开。v2 在 ARM 上相对 v1 额外加速 22%, 同时修

复了 v1 在 x86 上的 38% 性能倒退，使 FastScan 成为双平台一致的全局最优方案。perf 数据（IPC 0.96→1.71→1.94, L1D miss 2.70%→0.88%→0.65%）量化验证了从内存密集型到计算密集型的负载特征转变。

教训： (1) cache 友好的内存布局必须与访问模式（标量累加 vs SIMD gather）匹配；(2) 跨平台验证至关重要——OoO 能力差异可以完全改变优化策略的效果；(3) 对于内存密集型负载，消除不必要的内存 RMW 往往比提升计算并行度更有效。

项目代码见 Git 仓库，ARM 平台通过 `bash test.sh 1 1` 可完整复现。

A 附录 A：AI 辅助学习记录

本阶段使用 AI 辅助代码结构设计、SIMD intrinsics 调试和报告大纲生成。初始设计基于 `flat_scan.h` 的朴素 Flat Search，目标为 SIMD 加速的距离计算。

采纳的 AI 建议： (1) SIMD 阶段以 Flat 距离计算优化为基础，保留 scalar/auto/manual 三版本；(2) 使用 perf 采集硬件计数器；(3) PQ 实现覆盖 SDC 和 ADC；(4) 跨平台条件编译隔离 ISA intrinsics；(5) FastScan 核心优化方向为 LUT 量化 + block scan + 累加方式优化。

拒绝/延后的建议： (1) pshufb SIMD 查表——对 256-entry LUT 实测慢 15×；(2) OPQ 和 RaBitQ——超出本次强制范围；(3) 修改 test.sh/qsub.sh——违反课程要求。

实验验证： 所有 AI 建议均经双平台实验验证。关键发现（v1 在 x86 倒退、pshufb 比标量慢）修正了初始假设。AI 辅助学习，所有算法实现、实验设计、数据分析和报告撰写由作者主导完成。

B 附录 B：关键代码与 perf 输出

B.1 FastScan v2 核心代码 (ann_quant.h)

```
1 for (size_t b = 0; b < blocks; ++b) {
2     const uint8_t* block_codes = &codes[b * block_size * M];
3     for (size_t lane = 0; lane + 4 <= valid; lane += 4) {
4         const uint8_t *c0 = block_codes + lane * M;
5         const uint8_t *c1 = block_codes + (lane+1)*M, *c2 = c1+M, *c3 = c2+M;
6         int s0=0, s1=0, s2=0, s3=0; // REGISTERS
7         for (int part = 0; part < M; ++part) {
8             s0 += (int)qlut[part*256 + c0[part]];
9             s1 += (int)qlut[part*256 + c1[part]];
10            s2 += (int)qlut[part*256 + c2[part]];
11            s3 += (int)qlut[part*256 + c3[part]];
12        }
13        coarse[base+lane+0] = {-(float)s0, (uint32_t)(base+lane+0)};
14        coarse[base+lane+1] = {-(float)s1, (uint32_t)(base+lane+1)};
15        coarse[base+lane+2] = {-(float)s2, (uint32_t)(base+lane+2)};
16        coarse[base+lane+3] = {-(float)s3, (uint32_t)(base+lane+3)};
17    }
18 }
```

B.2 ARM test.sh 输出

```
$ bash test.sh 1 1
final ANN path: FastScan-ADC-M16-p1500-b64 build_time_sec=1.058
average recall: 0.99995
average latency (us): 2336.53
```

B.3 ARM perf 输出 (FastScan v2)

```
$ perf stat ./main --kernel fsadc-m16-p1500-b64 2000 3
cycles:      51,095,260,368
instructions: 99,256,058,438   IPC: 1.94
L1-dcache-load-misses: 0.65%   LLC-load-misses: 30.84%
```

B.4 x86 AVX2 汇编 (Flat-AVX 核心循环, MSVC /FACs)

```
vmovups ymm0, YMMWORD PTR [rax+rcx-32]    ; load a[d]
vmulps  ymm1, ymm0, YMMWORD PTR [rax-32]  ; a[d]*b[d]
vaddps  ymm2, ymm1, ymm2                  ; sum0 +=
; 4 accumulators: ymm2, ymm3, ymm4, ymm5
```

B.5 ARM NEON 汇编 (Flat-NEON 核心循环, GCC -O2 -S)

```
ldr  q0, [x1]          ; load 4 floats
fmla v4.4s, v0.4s, v1.4s ; fused multiply-add
; 4 accumulators: v4, v5, v6, v7 (unroll4)
```